

Full Length Article

Self-action distribution system (SADS): From neural network stability analysis to memory networks with low storage space cost [★]Fanglue Liu ^{a,b,c}, Ting Qian ^{d,*}, Yanhong She ^{d,*}^a School of Petroleum Engineering, Xi'an Petroleum University Xi'an, Shaanxi, 710065, PR China^b Engineering Research Center of Development and Management for Low to Ultra-Low Permeability Oil & Gas Reservoirs in West China (Xi'an Petroleum University), Ministry of Education, PR China^c Shaanxi Key Laboratory of Well Stability and Fluid & Rock Mechanics in Oil and Gas Reservoirs, PR China^d School of Science, Xi'an Petroleum University Xi'an, Shaanxi, 710065, PR China

ARTICLE INFO

Keywords:

Generalized upper bound
Memory mechanism
Lipschitz condition
Neural network stability

ABSTRACT

Currently, neural network code editing is employed directly, without any theoretical analysis of its stability. This practice can lead to network instability and unnecessary resource consumption. To address these issues, this paper makes the following innovative contributions. First, we introduce a method for evaluating the stability of neural networks. Since this evaluation does not require gradient computation, it is applicable to a wide range of complex architectures. By applying this method, algorithm optimization can be performed prior to training, which helps identify potential performance bottlenecks at an early stage. Second, building on the proposed evaluation method, we present an implementation of a memory network with favorable space complexity. Both theoretical analysis and experimental results confirm that this memory network can effectively represent symbols. Interestingly, the proposed memory mechanism aligns with certain functions of the human brain's temporal lobe, offering valuable insights for research areas such as brain-inspired computing and the development of in-memory computing chips. Finally, based on this memory network, we propose a novel neural network algorithm called the Self-Action Distribution System (SADS). Experimental validation demonstrates that SADS outperforms existing algorithms, confirming its effectiveness.

1. Introduction

Deep neural networks are also becoming more common in many scientific applications. However, there are still problems to be solved in many fields.

•In terms of network stability:

In the general application scenario of the neural network algorithm, a relevant algorithm is designed for a specific problem, and then the algorithm is trained on a data set and the convergence of loss is observed. The algorithm is adjusted again according to the convergence property and then these steps were repeated. However, due to the lack of theoretical discussion on the effectiveness of the algorithm, a lot of training resources will be wasted. In the background of global energy shortage (Lei et al., 2026; Lv et al., 2026), this “post-design-training” high-energy consumption trial-and-error model in the field of neural networks is inefficient and unsustainable.

In order to avoid the waste of unnecessary resources, in Bouchereau et al. (2023), the authors investigated the stability of deep neural networks in approximating physical functions and proposed that small

perturbations must be controlled and quantified. In 2010, in Szegedy et al. (2014), Szegedy-Goodfellow et al. proposed that the Lipschitz constant K^* corresponding to the neural network expressed the stability of the neural network. Later, K^* has been used in Couellan (2021), Gouk et al. (2021) and Couellan (2022) to improve the stability of neural networks. A more elaborate Lipschitz upper bound was discussed in Lee et al. (2020), Weng et al. (2018), Virmaux and Scaman (2018), Pintore and Després (2022), and Pintore and Després (2024). However, the above discussion was limited to the multilayer BP network. And it was difficult to expand to other network models, so it was necessary to discuss more general situations. Moreover, the above conclusions depend on the calculation of the gradient, increasing the difficulty of conducting the stability analysis of the algorithm.

•In terms of memory networks:

In Weston et al. (2015), Sukhbaatar et al. (2015) and Weston et al. (2016), the structure of the memory network and related applications were discussed. In Taha (2025), a new model was proposed that uses the hierarchical log tree structure for remote sequence modeling scenarios to efficiently store and retrieve past information. Using the

* Corresponding author.

E-mail addresses: qiant2000@126.com (T. Qian), yanhongshe@xsyu.edu.cn (Y. She).<https://doi.org/10.1016/j.neunet.2026.109635>

Received 20 March 2026; Received in revised form 6 September 2026; Accepted 11 September 2026

Available online 12 September 2026

0893-6080/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

characteristics of hierarchical algorithm and RNN algorithm, this method reduces the memory footprint and computational complexity from $O(n^2)$ to $O(\log(n))$. In Yuechen et al. (2026), A memory model based on generalized associative temporary encoding has been proposed. In complex tasks, multi-layer models develop overlapping, evidence, traces, and other representations related to the task. Under changes in the structure of sensory encoding, position scaffolds, or task parameters, the model reuses the learned representations and learns more quickly. According to the above discussion on network capacity, more neurons were needed to realize memory function. Therefore, the memory capacity needs to be optimized.

•In terms of network performance improvement:

In Jishnu and Cornelia (2023a), the nonlinear RvNN algorithm has been improved by segmenting input long sequences into short sequences and introducing the "recursion within recursion" (RIR) mechanism during the recursion process. By analyzing RIR, CRvNN(Continuous Recursive Neural Network)(Jishnu & Cornelia, 2019), OM(Order memory)(Sukhbaatar et al., 2019), S4D(stacked bidirectional S4D model implemented similarly to S4D-inv)(Albert et al., 2022), MEGA (Xuezhe et al., 2023), it was confirmed that the RIR algorithm further improves network performance compared to the original algorithm. Although the process of splitting the input can effectively avoid the gradient disappearance problem of the RvNN algorithm, this part of the algorithm has high time and space complexity. Therefore, other methods were needed to improve it.

Based on the literature and analysis above, several issues require investigation. Firstly, a stability analysis method for general neural networks should be discussed, and to streamline the analysis process, it is advisable to avoid using gradients. Secondly, the classification ability of general neural networks needs examination. Finally, an efficient memory model must be proposed.

The achievements obtained in this paper are as follows.

1. Properties of the activation functions *tanh*, *Sigmoid*, *Elu*, *ReLU*, and *leakReLU* are given. Based on the properties of the activation functions, whether neural networks are stable is discussed.

2. Based on the previous discussions, we further construct the ERIR model and the memory model. Subsequently, we conduct an in-depth demonstration to prove that the memory model has larger capacity and better stability.

3. Using the ERIR model and the memory model, we give a new self-action distribution system (SADS) model. The results of the experiment show that the SADS model can complete the learning task better than existing algorithms.

The neural network stability calculation method studied in this paper can be used to conduct similar explorations on other neural network algorithms. With this method, we can identify the theoretical performance bottlenecks of the network before conducting network training and parameter adjustment, and then make optimizations and improvements accordingly.

This measure can significantly reduce the consumption of computing resources during network training. On the other hand, the above-mentioned analysis process for theoretical performance bottlenecks does not require gradient calculation, effectively reducing the difficulty of theoretical analysis.

2. Preliminaries

Theorem 2.1. Guido et al. (2014) A max-out network with L layers of width n_0 and rank k can compute functions with at least $k^{L-1}k^{n_0}$ linear regions.

Theorem 2.1 describes the structure of the neural network of BP and determines the upper limit of its ability to partition linear regions in Guido et al. (2014).

According to Theorem 2.1, increasing the number of neurons in each layer of a neural network is much less effective than increasing the depth of the neural network.

Due to the structure of the Recurrent Neural Network (RNN), it can be roughly considered that an RNN is similar to a Back Propagation (BP) network with the number of hidden layers equal to the length of the input data. Therefore, for relatively long data, the RNN has a relatively large depth, which gives it strong classification ability. However, since RNNs cannot be parallelized and suffer from problems such as gradient vanishing and gradient explosion, it is difficult to effectively leverage the advantages of the RNN classification ability.

In view of this, reference Jishnu and Cornelia (2023a) focuses on the parallelization problem of recurrent neural networks (RNN). The algorithm proposed in this research precisely selects the split points in the sequence to recursively break down long sequences into multiple short sequences. Specifically, RNN operations are carried out independently on each short sequence first, and then the operation results of these short sequences are recursively combined. By means of this ingenious approach, the difficulties in parallelizing RNN networks can be effectively overcome, and the problems of gradient vanishing and gradient explosion can be significantly alleviated.

Definition 2.1. Walter (2020) Suppose that $(\mathcal{X}, \rho)$ is a metric space and T is a contraction mapping on $(\mathcal{X}, \rho)$. If there exists $0 < \alpha < 1$ such that for any $x, y \in \mathcal{X}$, we have $\rho(T(x), T(y)) \leq \alpha * \rho(x, y)$.

Theorem 2.2. Walter (2020) Suppose that $(\mathcal{X}, \rho)$ is a complete metric space and T is a contraction mapping on $(\mathcal{X}, \rho)$. Then there exists a unique fixed point $x_0 \in \mathcal{X}$ such that $T(x_0) = x_0$. Moreover, if an arbitrary initial point x_1 is chosen, then the sequence of points $x_{n+1} = T(x_n)$, $n \in \mathbb{N}^+$ will converge to x_0 .

In functional analysis (Walter, 2020), the Definition 2.1 and the Theorem 2.2 are generally known as the contraction mapping and the Banach fixed point theorem, respectively. Since the n -dimensional vector space is a complete metric space, Theorem 2.2 indicates that any specific symbol corresponds to a specific operation method from the perspective of artificial intelligence.

3. Stability analysis of neural networks

3.1. Properties of the activation functions

When analyzing the stability of neural networks, the calculation of gradients is often quite complex. To avoid this complexity, it is necessary to break down each component of the general neural network's forward computation process. The forward computation process can be decomposed into two parts: the linear transformation process and the calculation of nonlinear activation functions. The linear part naturally satisfies the sub-multiplicativity of norms, so the matrix product does not amplify the growth rate of the norm of the input vector. Therefore, the overall stability of the network is crucially dependent on the effect of the nonlinear activation function on the norm. This chapter will elaborate on the norm properties of common activation functions, such as *tanh*, *Sigmoid*, *Elu*, *ReLU*, and *leakReLU*, in order to analyze the stability of the network during the propagation process. In this paper, unless otherwise specified, f is used to represent the activation function mentioned above that is to be discussed.

•Property I

(1) There exist $L > 0$, for any $x \in \mathbb{R}, y \in \mathbb{R}$, we have $|f(x) - f(y)| \leq L * |x - y|$

(2) There exist $L > 0$, for any $x \in \mathbb{R}, y \in \mathbb{R}$, we have $|f(x + y)| \leq L * (|f(x)| + |f(y)|)$

(3) There exist $\epsilon \geq 0, L > 0$, for any $x \in \mathbb{R}$, we have $|f(x)| \leq L * |x| + \epsilon$

Proof. It is easy to prove for the activation functions *tanh*, *Sigmoid*, *Elu*, *ReLU*, and *leak ReLU*. In the following, take the function *Sigmoid* as an example to show the proof.

(1) The function *Sigmoid* is $f(x) = 1/(1 + \exp(-x))$. Therefore, the derivative of $f(x)$ can be calculated as $f'(x) = \exp(-x)/(1 + \exp(-x))^2 < 1/(1 + \exp(-x)) < 1$. So, according to Lagrange's Mean Value Theorem, we know that Property I(1) is valid.

(2) By the properties of proper fractions, when a positive number is added to both the numerator and the denominator, the value of fractions increases. So we know $f(x+y) = 1/(1 + \exp(-x-y)) \leq (1 + \exp(-x) + \exp(-y))/(1 + \exp(-x-y) + \exp(-x) + \exp(-y)) < (2 + \exp(-x) + \exp(-y))/(1 + \exp(-x-y) + \exp(-x) + \exp(-y)) = f(x) + f(y)$. Therefore, Property I(2) holds.

(3) Suppose that $F(x) = |f(x)| - |x| = 1/(1 + \exp(-x)) - |x|$, then $F(0) = 1/2 > 0$. Since the function *Sigmoid* is a bounded function, $F(x) < 0$ when $|x|$ is large enough. Thus, Property I(3) holds without an interval. When $x > 0$, $F(x)$ is monotonically decreasing. Therefore, according to the Zero Point Theorem, $F(x)$ has a unique zero point when $x > 0$. Similarly, it can be proven that $F(x)$ has a unique zero point when $x < 0$.

Thus, when $x \geq 0$, $F(x) = |f(x)| - |x| = f(x) - x$, then $F'(x) = \exp(-x)/(1 + \exp(-x))^2 - 1 < 0$. when $x < 0$, $F(x) = |f(x)| - |x| = f(x) + x$, then $F'(x) = \exp(-x)/(1 + \exp(-x))^2 + 1 > 0$. In conclusion, the maximum value of $F(x)$ is achieved at $x = 0$, and at this point $F(x) = 0.5$. From the above proof, it can be concluded that Property I(3) holds when f is the *Sigmoid* function. At this time, $\epsilon = 0.5$.

In fact, Property I can be extended to promote in the n -dimensional vector space. $\square$

•Property II

(1) There exist $L > 0$, for any $x \in R^n, y \in R^n$, we have $\|f(x) - f(y)\| \leq L * \|x - y\|$

(2) There exist $L > 0$, for any $x \in R^n, y \in R^n$, we have $\|f(x+y)\| \leq L * (\|f(x)\| + \|f(y)\|)$

(3) There exist $L > 0, \epsilon = \{\epsilon_1, \epsilon_2, \dots, \epsilon_n\}$, and $\epsilon_i \geq 0$, for any $x \in R^n$, we have $\|f(x)\| \leq L * \|x + \epsilon\|$

The norm in the above properties can be the norm L^1 or the norm L^2 .

Note1: It is particularly noteworthy that for some activation functions, the equality sign in parts of the above inequality property cannot be achieved. This characteristic of such activation functions will affect the model performance during the experiment and should be fully considered during model design and training.

Note2: Since data normalization is frequently performed in actual network construction, Property II(3) can be rewritten as: There exist $\epsilon > 0$, For any $x \in R^n$, we have $\|f(x)\| \leq \|x\| + \epsilon$

Note3: For simplicity of writing, all the activation functions in the following network are set to *Relu*, except for those with special explanations.

3.2. Stability analysis of some network

Since some commonly used activation functions satisfy the inequality properties introduced in Section 3.1 and the norm of matrix multiplication satisfies sub-multiplicativity, it is possible to discuss the impact on the output when the input undergoes small changes, that is, the stability of Bounded-Input Bounded-Output(BIBO). In some cases (for instance, in LSTM networks), the following inequality technique might be used.

$$\begin{aligned} & \|A * B - C * D\| \\ &= \|A * B - A * D + A * D - C * D\| \\ &\leq \|A * B - A * D\| + \|A * D - C * D\| \\ &= \|A * (B - D)\| + \|(A - C) * D\| \\ &\leq \|A\| * \|B - D\| + \|D\| * \|A - C\| \end{aligned}$$

During the BIBO network stability analysis process, terms similar to $\|f(x) - f(y)\| \leq M * \|x - y\| + C(x, y)$ may occur. Therefore, the

stability discussed in this paper cannot simply be regarded as Lipschitz stability.

It should be noted that the value of L and whether the equal sign can be taken in the above inequality has a significant impact on the precision of the analysis results. Thus, in general, the inequality of the analysis cannot be expanded on the basis of the boundedness of the independent variable.

3.2.1. Stability analysis of BP network

Suppose that the structure of the *BP* network is as shown in Fig. 1, then the stability of this network can be analyzed using the properties of the activation function described in Section 3.1.

Suppose $x_i = (x_1^i, x_2^i, \dots, x_{n_i}^i)$, $i \in 1, 2, \dots, m$ and the change of x_m is caused by the slight change $\tilde{x}_1$ of x_1 , then $\|x_m - \tilde{x}_m\| = \|f(W_{m-1} * x_{m-1}) - f(W_{m-1} * \tilde{x}_{m-1})\| \leq \|W_{m-1}\| * \|x_{m-1} - \tilde{x}_{m-1}\| = \|W_{m-1}\| * \|f(W_{m-2} * x_{m-2}) - f(W_{m-2} * \tilde{x}_{m-2})\| \leq \|W_{m-1}\| * \|W_{m-2}\| * \|x_{m-2} - \tilde{x}_{m-2}\| \dots \leq \prod_{i=1}^{m-1} \|W_i\| * \|x_1 - \tilde{x}_1\|$

From the above derivation process, it can be clearly seen that the stability of the *BP* network depends on the product of two types of coefficient matrix norms: one is the product of the norms of coefficient matrices greater than 1, and the other is the product of the norms of coefficient matrices less than 1. As the number of network layers increases and the depth of the network deepens, the cumulative effect of these two types of matrix norm products will be significantly amplified. For coefficient matrices with norms greater than 1, their norm product will increase sharply with the increase in network depth, causing the problem of gradient explosion; while for coefficient matrices with norms less than 1, their norm product will rapidly approach zero as the network depth increases, leading to the problem of gradient disappearance. Both of these situations will seriously undermine the stability of the *BP* network, making it difficult for the network to converge or causing a very slow convergence speed, thereby affecting the training effect and performance of the network.

It can be deduced from this chapter that when the Lipschitz constant of the base network is too small or stable, the residual network can play a role in correcting the network. However, when the Lipschitz constant of the basic network is unstable or too large, the addition of the residual network does not contribute to improving the network performance.

3.2.2. Stability analysis of RNN network

Suppose that the structure of the *RNN* network is as shown in Fig. 2, then the stability of this network can be analyzed using a similar method as described in Section 3.2.1.

Suppose that the change of x_n is caused by the slight change $\tilde{x}_m$ of x_m , $m \in \{1, 2, 3, \dots, n\}$, then $\|h_{n+1} - \tilde{h}_{n+1}\| \leq \|f(h_n) - f(\tilde{h}_n)\| \leq \|h_n - \tilde{h}_n\| \leq \|f(U * h_{n-1} + V * x_n) - f(U * \tilde{h}_{n-1} + V * x_n)\| \leq \|U\| * \|h_{n-1} - \tilde{h}_{n-1}\| \leq \|U\|^{n-m} * \|V * x_m - V * \tilde{x}_m\| \leq \|U\|^{n-m} * \|V\| * \|x_m - \tilde{x}_m\|$

From the above proof, it can be observed that the stability of the *RNN* network is closely related to two key factors. The first factor is the value of $n - m$, which represents the distance between the position of the change value and the length of the data. The second factor is whether the norm of the coefficient matrix U is greater than 1.

From the above analysis process, it can be seen that since the method proposed in this paper bypasses the gradient calculation process, the amount of work required to complete the analysis has decreased significantly compared to the original analysis method.

4. Improvement of RIR network

In this chapter, the design flaws of the *RIR* model are analyzed using the methods in Section 3, and the corresponding optimization solutions are provided.

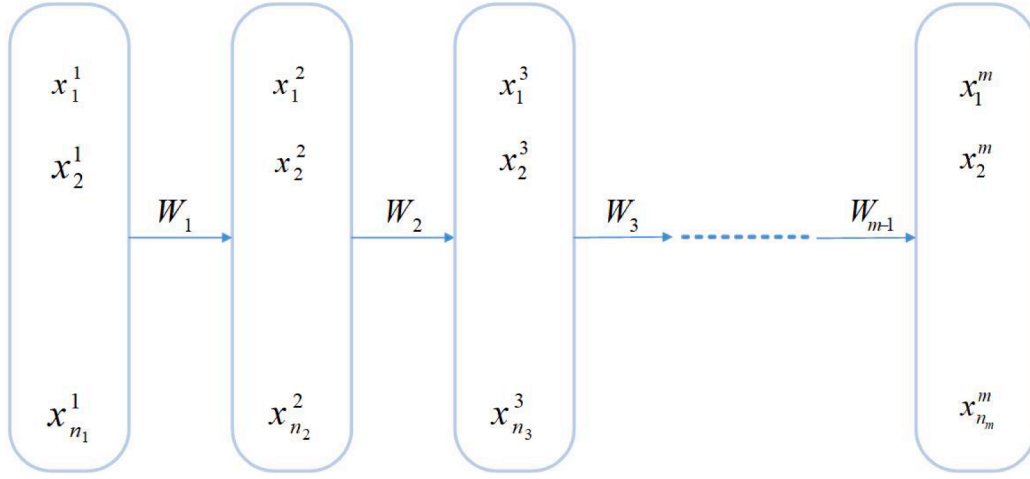


Fig. 1. BP network structure.

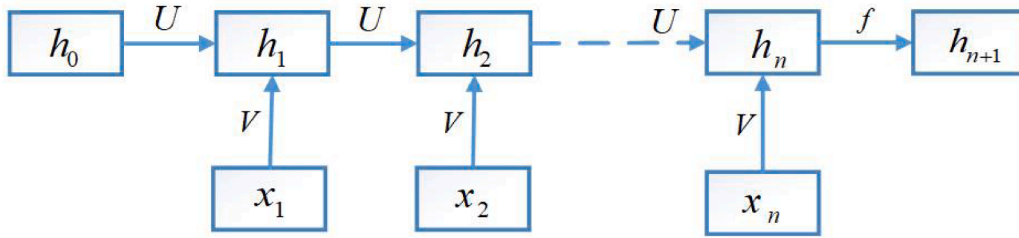


Fig. 2. RNN network structure.

4.1. An example

For convenience, all the discussions in this section are based on the following RIR model using the activation function *ReLU* (the activation function is denoted f in the following derivation).

Suppose that the training samples in the dataset are a sequence of $\{x_i = (x_{i1}, x_{i2}, x_{i3}, x_{i4}) : i \text{ from } 1 \text{ to } n \in N^+\}$, corresponding to the label $\{y_i : i \text{ from } 1 \text{ to } n \in N^+\}$, and use the basic RIR model for prediction. The specific execution process is shown in Fig. 3.

In Fig. 3, the RIR model divides one of the sample data $x_i = (x_{i1}, x_{i2}, x_{i3}, x_{i4})$ into two subsequences (x_{i1}, x_{i2}) and (x_{i3}, x_{i4}) , and inputs the two subsequences into the RNN model to obtain x_{i5} and x_{i6} , respectively. Finally, the sequence consisting of x_{i5} and x_{i6} is fed into the RNN model to obtain the final result $x_{i7} = y_i$.

Note1: For convenience in subsequent discussions, the RIR model is simplified here. In order to achieve better performance, the RIR model in Jishnu and Cornelia (2023a) selects better points to segment the training sequence using an algorithm.

4.2. Stability analysis of RIR network

For the example described in Section 4.1, assuming that x_{i3} undergoes a slight change and becomes $\widetilde{x}_{i3}$, the corresponding output result becomes $\widetilde{x}_{i7}$. Then, according to the properties of the activation function in Section 3.1, the following inequality holds.

$$\begin{aligned}
 & \|x_{i7} - \widetilde{x}_{i7}\| \\
 &= \|f(h_6) - f(\widetilde{h}_6)\| \\
 &\leq \|h_6 - \widetilde{h}_6\| \\
 &= \|f(U * h_5 + V * x_{i6}) - f(U * h_5 + V * \widetilde{x}_{i6})\| \\
 &\leq \|V * x_{i6} - V * \widetilde{x}_{i6}\| \\
 &= \|V * (x_{i6} - \widetilde{x}_{i6})\| \\
 &\leq \|V\| * \|x_{i6} - \widetilde{x}_{i6}\| \\
 &= \|V\| * \|f(U * h_3 + V * x_{i4}) - f(U * \widetilde{h}_3 + V * x_{i4})\| \\
 &\leq \|V\| * \|U * h_3 - U * \widetilde{h}_3\|
 \end{aligned}$$

$$\begin{aligned}
 &\leq \|V\| * \|U\| * \|h_3 - \widetilde{h}_3\| \\
 &= \|V\| * \|U\| * \|f(U * h_0 + V * x_{i3}) - f(U * h_0 + V * \widetilde{x}_{i3})\| \\
 &\leq \|V\|^2 * \|U\| * \|x_{i3} - \widetilde{x}_{i3}\|
 \end{aligned}$$

It is easy to see that $\|V\|^2 * \|U\|$ in the above inequality is the Lipschitz constant of the neural network model in the Banach space, which described the stability of the neural network according to the discussion in Szegedy et al. (2014). As can be known from the above derivation process, in the RIR model, the change in the result is determined by the change in the input, the position where the change occurs, and the norm of the weight matrix of the neural network. As shown in Fig. 4, in the results derived described above, where the power of $\|U\|$ is the sum of the number of elements in each level after varying the input, the power of $\|V\|$ is the number of levels in which the input changes.

Therefore, when the power of $\|U\|$ and $\|V\|$ is too large, it will lead to network failure. The RIR algorithm has significant limitations regarding the number of elements in each layer and the number of layers during the layering process, which, in turn, restricts the maximum length of sequences that the model can handle.

4.3. The improvement of the RIR algorithm

According to the derivation process in Section 4.2, the main drawback of the RIR algorithm is that the power of $\|U\|$ and $\|V\|$ is higher when dealing with longer sequences. Therefore, the following mechanisms are introduced in the Enhanced RIR algorithm (ERIR algorithm).

Network Structure Replacement Mechanism: given that the BiLSTM algorithm has characteristics similar to skip connections, we replace traditional RNN networks with BiLSTM networks at each layer. This replacement can effectively improve the network's ability to process sequential information. Since BiLSTM can consider both the forward and backward information of the input sequence, it can capture long-range dependencies more effectively than RNN.

Residual Network Mechanism: On the basis of the original calculation results, we superimpose a residual calculated by a single

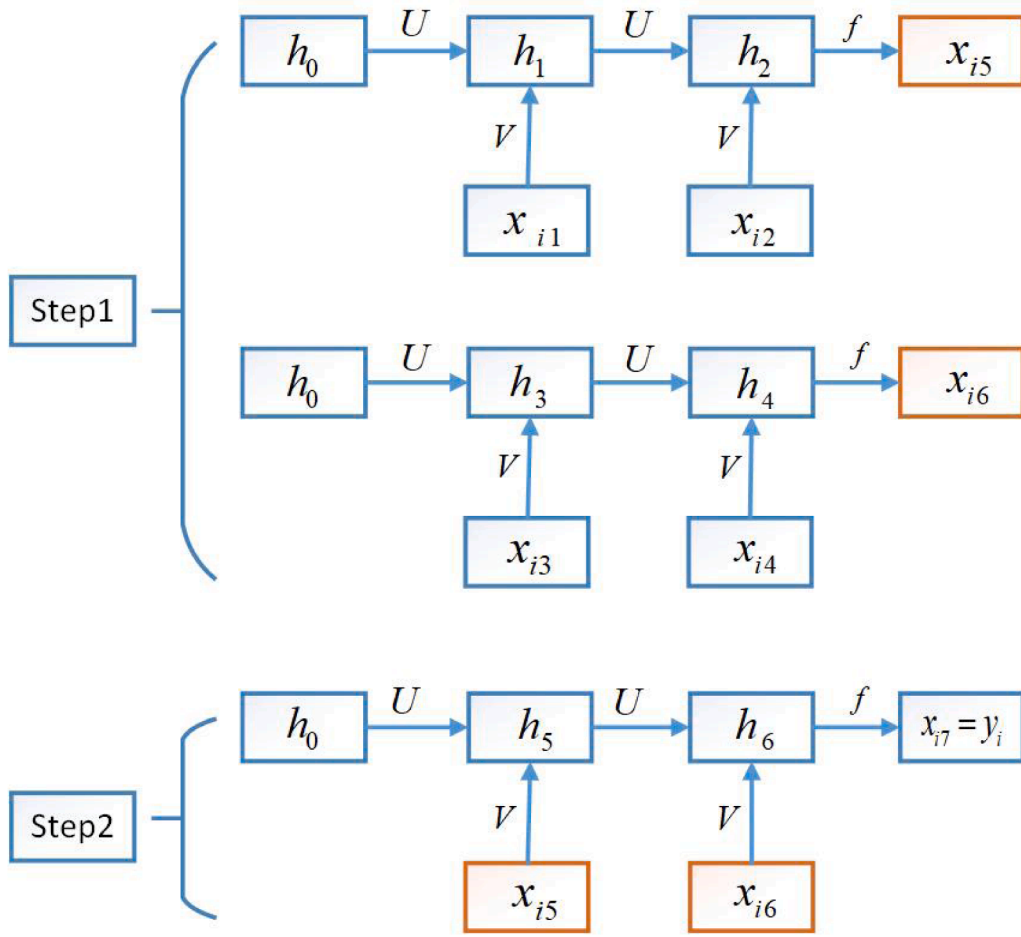


Fig. 3. RIR process stream.

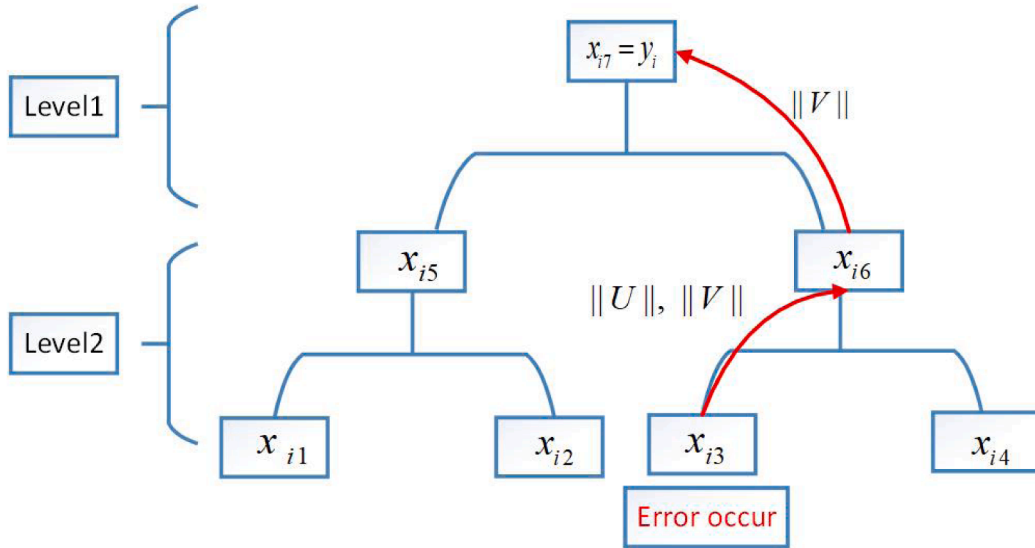


Fig. 4. RIR error spread path.

hidden-layer BP network. When the basic network composed of multiple levels of BiLSTM has difficulty in effectively controlling the stability, the residual network can come into play to further optimize the network performance. In this way, the network can adaptively adjust the output to compensate for possible deficiencies in the basic network.

CrossLen Mechanism: To effectively reduce edge loss caused by split strings, we adopt the CrossLen mechanism. This mechanism can reasonably process the split strings, reducing the information loss and errors at the edge parts of the strings due to the splitting operation, thereby improving the overall processing effect.

Two-Tower Mechanism: In order to significantly improve computational efficiency, we introduce the two-tower model. This model processes the input data separately and then fuses the features at the end. It can handle different tasks in parallel, avoiding the serial computing bottlenecks that may exist in traditional models, thus greatly increasing the computing speed and efficiency.

Non-Functional Mechanism: In order to effectively address the non-functional issues present in the data, we adjusted the output mechanism of the model, which would output a vector containing all possible values. And when calculating the loss, we select the output result that is closest to the label to calculate the error and optimize the loss.

Optimal parameter estimation: First, we calculated the maximum sequence length in the training data set. Then, we conducted the BIBO stability analysis for the ERIR algorithm using a process similar to that described in Section 4.2. Finally, based on the maximum sequence length, we determined the range of parameter values to ensure that no higher-order terms or a series of multiplying matrix norms would be encountered in the BIBO stability analysis of the ERIR algorithm.

The ERIR algorithm is described in Algorithm 1 and Algorithm 2.

To improve computational efficiency, first compute the residual results of the original data, then recursively merge the input data into chunks where $xNum$ is less than or equal to $mLen$, accordingly increasing $batch$. Subsequently, based on the results of algorithm 1, calculate the ERIR result using algorithm 2.

Algorithm 1 Calculate x , $oriBatch$, $resRst = firstTower(x)$.

Input: x is input tensor of form $[batch, xNum, xDim]$
Input: $hDim > 0$
Input: $mLen > 0, sNum > 0, 0 < crossRate < 1$

- 1: $batch, xNum, xDim \leftarrow x.shape$
- 2: $oriBatch \leftarrow batch$
- 3: $outRes \leftarrow Bp(xDim, hDim, labelDim)$
- 4: $resRst = outRes(x)$
- 5: **while** $x.shape[1] > mLen$ **do**
- 6: $batch, xNum, xDim \leftarrow x.shape$
- 7: **if** $mod[xNum, sNum] \neq 0$ **then**
- 8: $err \leftarrow [xNum/sNum] * sNum + sNum - xNum$
- 9: $padData \leftarrow zeros[batch, err, xDim]$
- 10: $x \leftarrow Add[x, padData, dim \leftarrow 1]$
- 11: **end if**
- 12: $x \leftarrow x.view[batch * sNum, xNum / sNum, xDim]$
- 13: **end while**
- 14: **return** $x, oriBatch, resRst$

In Algorithm 1, the original batch is stored as **oriBatch** at step 2. In steps 3 - 4, the residual results of the original data are calculated. In steps 5 - 13, the $xNum$ is gradually reduced and the $batch$ is correspondingly increased.

In Algorithm 2, the parameter **isCalERir** prevents the situation where the ERIR calculation cannot be performed because the data are too short in steps 2 and 5. In steps 7 - 11, the data obtained by calculating the results of the **crossLenRate** ratio of the data in the junction part are merged with the original data. In step 13, the data are calculated using BiLSTM. In steps 14 - 16, the results of the data calculation **crossLenRate** ratio are deleted. In step 19, **batch** and **xNum** are gradually restored until **batch** equals **oriBatch**. In step 23, add the residual network calculation results to the original calculation results.

Since the ERIR algorithm adds the single hidden layer BP and BiLSTM mechanism, the new network derived from 4.2 does not easily lead to the problem of gradient disappearance at recursively deeper depths. Therefore, the process of selecting the appropriate segmentation points in the ERIR algorithm becomes less essential. To reduce the

Algorithm 2 (ERIR):

Calculate $y = ERIR(x, oriBatch, resRst)$.

Input: x is input tensor of form $[batch, xNum, xDim]$

Input: $hDim > 0$

Input: $mLen > 0, sNum > 0, 0 < crossRate < 1$

- 1: $x, oriBatch, resRst \leftarrow firstTower(x)$
- 2: $isCalERir \leftarrow false$
- 3: $batch, xNum, xDim \leftarrow x.shape$
- 4: **while** $[oriBatch \neq batch]$ or $[not isCalERir]$ **do**
- 5: $isCalERir \leftarrow true$
- 6: $crossLen \leftarrow crossRate * x.shape[1]$
- 7: **if** $crossLen > 0$ **then**
- 8: $temp \leftarrow Add[x, x[:, 0 : crossLen, :], dim \leftarrow 1]$
- 9: $temp \leftarrow Add[x[:, crossLen : , x.shape[1] - crossLen - 1 : , :], temp, dim \leftarrow 1]$
- 10: **else**
- 11: $temp \leftarrow x$
- 12: **end if**
- 13: $out, hn, cn \leftarrow BiLSTM(temp)$
- 14: **if** $crossLen > 0$ **then**
- 15: $out \leftarrow out[:, crossLen : out.shape[1] - crossLen - 1, :]$
- 16: **end if**
- 17: $x \leftarrow out$
- 18: $batch, xNum, xDim \leftarrow x.shape$
- 19: **if** $batch > oriBatch$ **then**
- 20: $x \leftarrow x.view[batch/sNum, xNum * sNum, xDim]$
- 21: **end if**
- 22: **end while**
- 23: **return** $x + resRst$

consumption of GPU memory, we removed the point selection process from the RIR model.

5. Mask-based memory network

5.1. Memory network construction

As shown in the results of Section 4.2, when the powers of $\|U\|$ and $\|V\|$ are relatively high, the calculation results are likely to fail. Especially when $\|U\| < 1$ and $\|V\| < 1$, the network will be unable to respond effectively to changes in the input, that is, the gradient vanishing phenomenon will occur.

Since what we need is a predictive model, it is generally believed that this is a phenomenon that should not be present in training networks.

However, if we want to build a memory model, the gradient vanishing can actually be a beneficial phenomenon. When $\|U\| < 1$, $\|V\| < 1$, and the powers of $\|U\|$ and $\|V\|$ are relatively high, then the inequality of $\|RIR(x) - RIR(y)\| \leq L * \|x - y\|$, $L < 1$ holds. According to Definition 2.1, the currently model is a contraction mapping in a Banach space. Therefore, according to Theorem 2.2, this neural network has a unique fixed point. This provides us with a new design concept: the memory content is represented by an almost constant output induced by different mask vectors.

However, memorizing a single vector using all the parameters of the whole network (which is basically equal to the number of elements of U plus the number of elements of V) would waste a lot of resources. Therefore, it is necessary to consider the method of achieving memory using only specific parts of the weights. Based on these discussions, an implementation method for a memory network can be obtained as follows.

Firstly, refer to the flow chart in Fig. 5, the input data is fed into a BP network to generate the mask vectors M_{iU} for the weight matrix

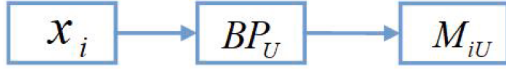


Fig. 5. Generation of memory mask.

of the ERIR algorithm. These mask vectors determine neurons, which need to be enabled for this computation.

Note1: Since the function *Sigmoid* satisfies the property $\forall x \in \mathbb{R}^n, y \in \mathbb{R}^n$ st. $\| \text{Sigmoid}(x) - \text{Sigmoid}(y) \| < \| x - y \|$, in the BP network for mask generation, compared to the functions *ReLU* and *leakReLU*, choosing the function *Sigmoid* as the activation function can bring the values in mask vectors closer to 0 or 1.

Secondly, refer to the flow chart in Fig. 6, when calculating RIR or ERIR, the weight matrix used is the new matrix obtained by multiplying the mask vector calculated in the process of Fig. 5 and the original weight matrix.

Note1: In the calculation process of Fig. 6, a calculation similar to $M_{iU} * U$ means that the vector M_{iU} is first extended to have the same number of rows as the matrix U . Then, each element of the extended M_{iU} is multiplied by the corresponding element of U to obtain a matrix. The resulting matrix has the same number of rows and columns as U .

5.2. Proof for the construction of memory networks

To illustrate the rationality of this mechanism design, we should introduce a series of theorems. **Theorem 5.1** proves that, under appropriate metrics, the similarity of the input vectors can guaranty the similarity of the mask vectors, thus ensuring the continuity of the memory extraction process for similar inputs. Furthermore, **Theorem 5.2** indicates that, under specific assumptions, if two mask vectors are sufficiently similar, the differences of the corresponding input vectors can be controlled. And **Theorem 5.3** shows that minor changes in the input will hardly be reflected in the memory results of the output.

Theorem 5.1. Assuming that all elements of the input data set $X \subseteq \mathbb{R}^n$ have been normalized, the mapping $M_x = f(Wx)$ from X to X is a neural network BP without hidden layers as defined above, the activation function being *Sigmoid*. Select the norms of the sample set and the label set as the 2-norm. Then, for any x, y belonging to X , if $d(x, y)$ approaches 0, then $d(M_x, M_y)$ approaches 0.

Proof. From the discussion in Section 3.2.1, it is obvious that the theorem holds. $\square$

Theorem 5.2. Assuming that all elements of the input data set $X \subseteq \mathbb{R}^n$ have been normalized, the mapping $M_x = f(Wx)$ from X to X is a neural network BP without hidden layers as defined above, the activation function being *Sigmoid*. Select the norms of the sample set and the label set as the 2-norm. If all elements in W follow the distribution $\mathcal{N}(0, 1)$, and $d(M_x, M_y) = 0$, then $P(d(x, y) = 0) = 1$.

Proof. Since W is a square matrix and the *Sigmoid* function is an injection, when W is full rank, according to the theory of linear equations, for any x and y , $d(M_x, M_y) = 0$ equivalent to $f(Wx) = f(Wy)$, then $x = y$ holds. Thus, the original proposition is obviously true in this case. Additionally, since the elements in W follow a distribution of $\mathcal{N}(0, 1)$, from Terrence (2012), it is known that $P(|W| = 0) = 0$, which also confirms the validity of the original proposition. $\square$

Theorem 5.3. Suppose that the memory network architecture $y = F(x)$ is as shown in Fig. 5 and Fig. 6. For any $X \subseteq \mathbb{R}^n$ and $x_i, \tilde{x}_i \in X$, if $\| x_i - \tilde{x}_i \| \rightarrow 0$, then $\| F(x_i) - F(\tilde{x}_i) \| \rightarrow 0$.

Note: Within the proof of Theorem 5.3, Frobenius norm $\| \cdot \|_F$ is adopted for matrices, while vectors keep the standard Euclidean L_2 -norm $\| \cdot \|_2$. If the 2-norm is to be used entirely, the above proof process needs to be revised, but this will not affect the correctness of the theorem.

Proof. Suppose that the memory network architecture is as shown in Fig. 5 and Fig. 6. The x_{i1} undergoes a slight change and becomes $\tilde{x}_{i1}$, and the corresponding output result becomes $\tilde{x}_{i7}$. Then, according to the properties of the activation function in Section 3.1, it is easy to prove that the following inequality holds.

$$\begin{aligned} & \| x_{i7} - \tilde{x}_{i7} \| \\ & \leq \| U \| * \| h_{i5} \| * \| M_{iU} - \widetilde{M_{iU}} \| + \| U \|^2 * \| \widetilde{M_{iU}} \| * \| h_{i0} \| * \| \\ & M_{iU} - \widetilde{M_{iU}} \| \\ & + \| U \|^2 * \| \widetilde{M_{iU}} \| * \| V \| * \| h_{i1} \| * \| M_{iU} - \widetilde{M_{iU}} \| + \| U \|^3 * \| \\ & \widetilde{M_{iU}} \|^2 * \| V \| * \| h_{i0} \| * \| M_{iU} - \widetilde{M_{iU}} \| \\ & + \| U \| * \| \widetilde{M_{iU}} \|^2 * \| V \|^2 * \| x_{i1} - \tilde{x}_{i1} \| \\ & + \| V \| * \| U \| * \| h_{i3} \| * \| M_{iU} - \widetilde{M_{iU}} \| + \| V \| * \| \widetilde{M_{iU}} \| * \| U \|^2 * \| \\ & h_{i0} \| * \| M_{iU} - \widetilde{M_{iU}} \| \end{aligned}$$

Due to the fact that in the training process there exists a value H for any i and j , it holds that $\| h_{ij} \| < H$. Therefore, the above inequality can be simplified as follows.

$$\begin{aligned} & \| x_{i7} - \tilde{x}_{i7} \| \\ & \leq \sum_{i=1}^2 (\| U \| * \| \widetilde{M_{iU}} \|^{i-1} * H * \| M_{iU} - \widetilde{M_{iU}} \|) + \sum_{i=1}^2 (\| U \|^{i+1} * \| \\ & \widetilde{M_{iU}} \| * \| V \| * H * \| M_{iU} - \widetilde{M_{iU}} \|) \\ & + \| U \| * \| \widetilde{M_{iU}} \|^2 * \| V \|^2 * \| x_{i1} - \tilde{x}_{i1} \| \\ & + \sum_{i=1}^2 (\| V \| * H * \| U \| * \| \widetilde{M_{iU}} \|^{i-1} * \| M_{iU} - \widetilde{M_{iU}} \|) \end{aligned}$$

Due to the occurrence of gradient vanish in the network, therefore, $\| U \| * \| \widetilde{M_{iU}} \|$ is less than 1. So, it can be concluded that as the sequence lengthens, the coefficient of $\| M_{iU} - \widetilde{M_{iU}} \|$ approaches zero. Also, since $\| x_i - \tilde{x}_i \| \rightarrow 0$, it can be concluded that $\| x_{i7} - \tilde{x}_{i7} \| \rightarrow 0$. As a result, a slight change in M_{iU} will not affect the final output result of the memory network.

Since the proof process of Theorem 5.3 can be extended to the general case. Therefore, a small change in x_i will not affect the output of the memory network. $\square$

Based on the above proof, it can be further observed that if variables x_1 and x_2 are input into the model separately and the masks m_1 and m_2 generated by them only have a few conflicting positions, then there is almost no interference between the output results of x_1 and x_2 . As the amount of input data continues to increase, the values in the mask vector will gradually undergo significant changes, ultimately leading to fundamental changes in the model's output results and subsequently causing the memory to be forgotten.

5.3. Rationality test for the construction of memory networks

To verify the actual performance of this memory model shown in Figs. 5 and 6, we conducted the experiments using random numbers and obtained the results presented in Figs. 7 and 8. The experimental process is as follows:

Step1: Model Training

(1) Data Generation: Use a random number generator to create the training data set x .

(2) Model Training: The generated data are substituted into this memory model for standard training until the model converges.

(3) Gradient Tracking: Output the trend of gradient changes for each parameter during the training process. The test results are shown in Fig. 7.

Step2: Single-point Stimulation and Memory Tracking(Performed Twice)

(1) In the input vector space, randomly select a fixed coordinate position i .

(2) Keep the values of all other coordinates in the input vector unchanged and only increase the value of the coordinate i , x_i , from its original value at a fixed step size.

(3) After each increment, input the current vector into the trained memory model and record the L^2 norm of its internal memory vector. The test results are shown as the curve 1 **PointTest1** and the curve 1 **PointTest2** in Fig. 8.

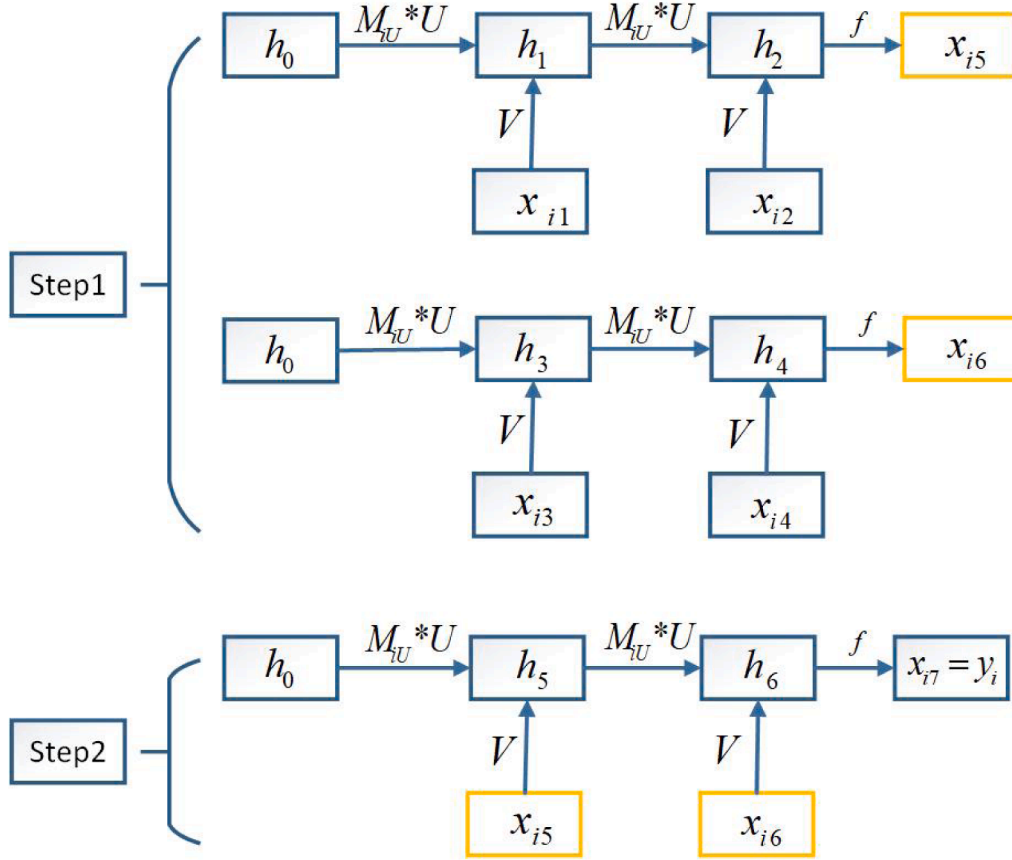


Fig. 6. Generate memory result.

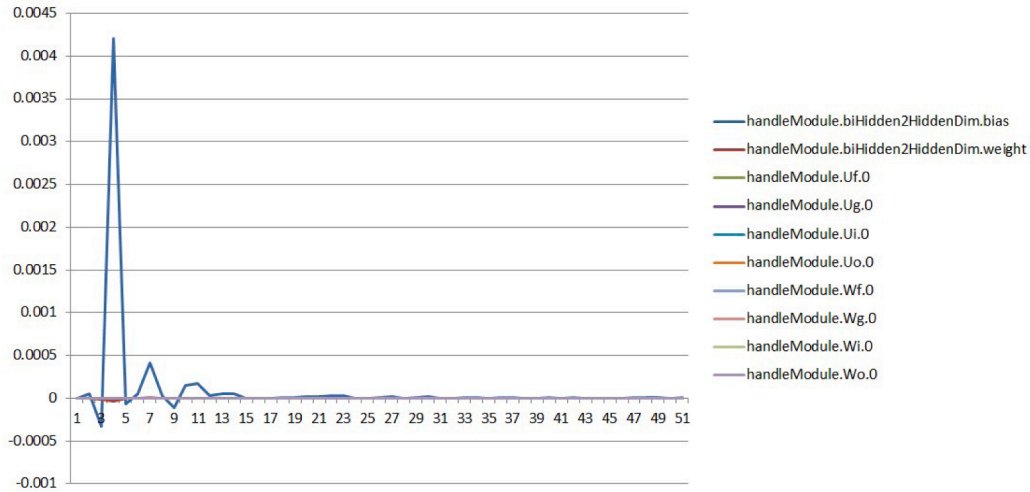


Fig. 7. Generate memory result.

Step3: Analysis of Double-Point Combined Stimulation and Synergy effect(Performed Twice)

(1) Randomly select a new coordinate $j(j \neq i)$ and form a stimulus combination i, j with the coordinate i selected in the first stage.

(2) Repeat the operation of the second stage, increasing the values of coordinate i and coordinate j , and recording the norm of the memory vector at each step. The test results are shown as the curve **2 PointTest1** and the curve **2 PointTest2** in Fig. 8.

Step4: Output Test Results

(1) Output the curve of the norm change of the single-point stimulus of the coordinate i .

(2) Output the curve of the norm variation of the stimulus coordinates j (when the coordinate i is fixed).

(3) Analyze inhibitory, enhancement, or nonlinear modulation effects demonstrated in the above experiment.

The test results for this memory model are shown in Fig. 7 and 8. Fig. 7 shows the results obtained by tracking the average gradients of some parameters during the model training process.

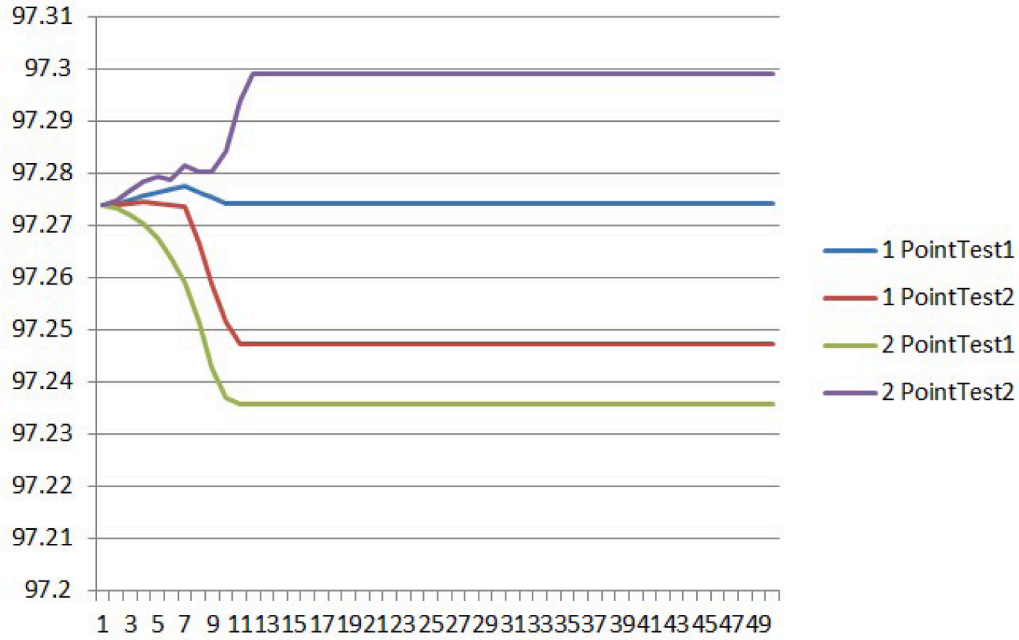


Fig. 8. Generate memory result.

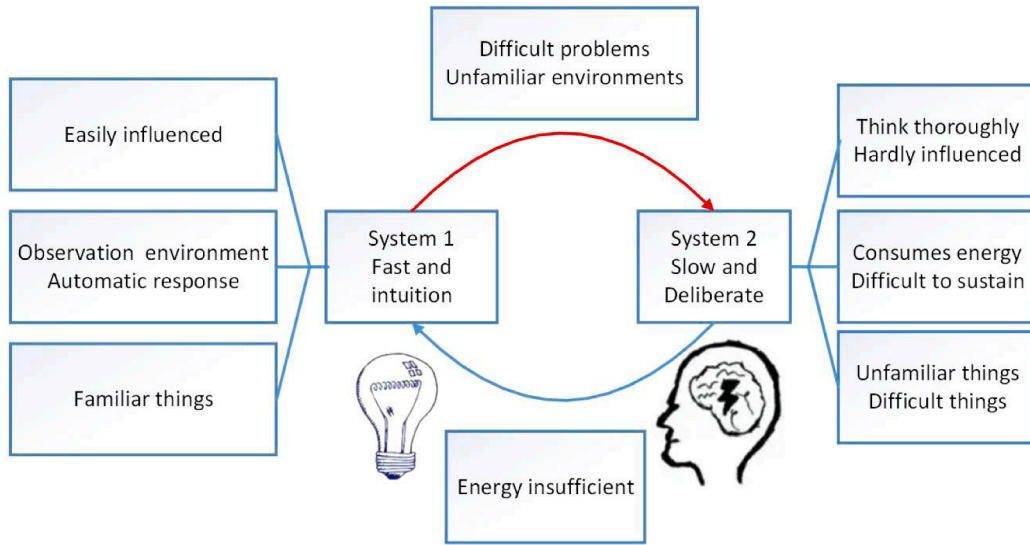


Fig. 9. Fast and slow stream in human brain.

Through the random perturbation test and the theorems introduced in this section, we observed some key phenomena of this memory model.

(1) The Mask mechanism contributes to the occurrence of gradient disappearance.

The Mask mechanism makes the occurrence of gradient vanishing more likely during the training of networks. This is beneficial for the training of deep memory units, indicating that the Mask mechanism plays a role in gradient regulation and information flow control to a certain extent.

(2) The model is relatively insensitive to input perturbations.

Based on the above experimental results and according to Theorem 5.1, it can be concluded that when minor input perturbations cause numerical conflicts at a few positions among different masks, their impact on the final output result is relatively limited.

(3) There is an isolation phenomenon in the memory between samples input at different times.

If part of the mask in the above memory model cannot continue to be trained for some reason, then the memory network cannot be used to store new memories. However, the retrieval of existing memories will not be affected.

(4) This mask transformation achieves a local consistency mapping between the input and the output.

Let the input vectors x_1 and x_2 belong to R^n . If the components of both vectors in a certain index set I are close, that is, $\|x_1 - x_2\|_I \rightarrow 0$, where $\|\cdot\|_I$ represents the norm of the sub-vector formed by the differences of the components of index i belonging to I . Then according to the matrix block multiplication rule in linear algebra, the mask vectors m_1 and m_2 generated by x_1 and x_2 satisfy $\|m_1 - m_2\|_J \rightarrow 0$. Here, J are the sub-indexes corresponding to I under the transformation. So, the mask transformation ensures that when the input components tend to be consistent in I , the output also tends to be consistent in J .

Based on the proof of the above theorem and the experimental results, assuming U is a matrix of dimension $m \times m$ and V is a ma-

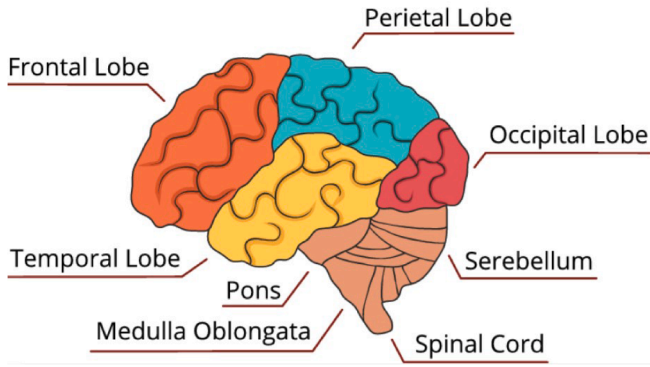


Fig. 10. Human brain regions.

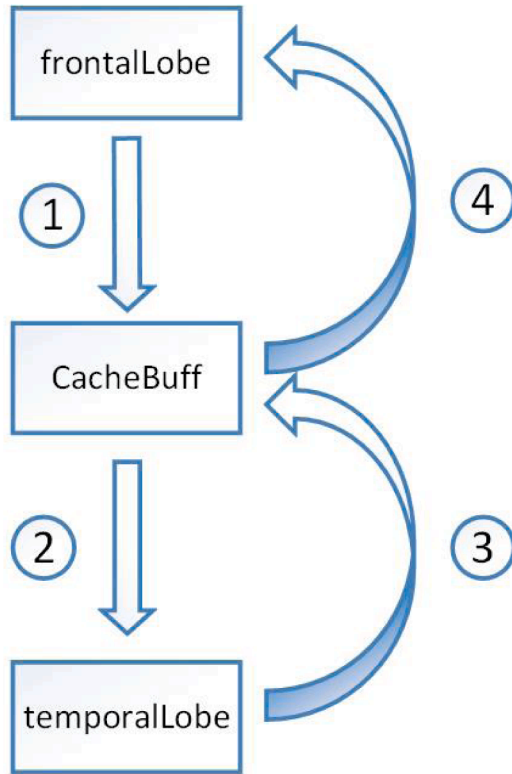


Fig. 11. SADS network process stream.

trix of dimension $m * n$, since there are $O(2^m)$ different mask vectors, we have memorized at most $O(2^m)$ vectors using only the weights of $O(m * m + m * n)$. Therefore, this model greatly exceeds the memory capacity of existing models by calculation. According to the processing flow in Fig. 6, the BiLSTM process in the ERIR algorithm (Algorithm 1 and Algorithm 2) can be rewritten accordingly. In subsequent discussions, we refer to this model as the MemERIR model.

Interestingly, if we assume that the temporal lobe in the human brain is constructed following the concept of the MemERIR model, and the part of the mask in the above memory model cannot continue to be trained for some reasons, then the memory network cannot be used to store new memories. However, the retrieval of existing memories will not be affected. This is similar to the symptoms mentioned in Suzanne (2018).

6. Self-action distribution system model

As shown in Fig. 9, there are System 1 and System 2 mechanisms in the human brain (Daniel, 2011).

In the human cognitive system, there exist two distinct operating modes, namely System 1 and System 2. System 1 is mainly responsible for fast processing of familiar information. Since these matters are already highly familiar to the brain, it can cache a large number of intermediate results during processing and quickly recall them when needed. This mechanism not only ensures the accuracy of the processing results but also significantly saves the energy consumed by the brain in handling the matters, while greatly improving the reaction speed. System 2, on the other hand, focuses on making precise considerations of matters. When dealing with complex, unfamiliar or highly accurate matters, System 2 is activated. It guides the brain to conduct a comprehensive and detailed analysis of the matter. Through this in-depth thinking and weighing, the accuracy of handling the issues can be effectively improved.

Based on the structure of the human brain shown in Fig. 10, the ERIR model derived in Section 4.3 and the MemERIR model derived in Section 5.1, we construct the self-action distribution system (SADS) model composed of the parietal lobe, the temporal lobe, and the CacheBuff.

Based on the discussion above, we introduce the mechanism shown in Fig. 11.

1. Independent computing components of **parietalLobe** and **temporalLobe** are constructed using the ERIR model or the MemERIR model.
2. The **cacheSize** length **cacheBuff** is used to store the intermediate results. This cache employs the First-In-First-Out (FIFO) replacement strategy: when the cache is full, the system will remove the first entered data to make room for new data.
3. When data are inputted into the network, they will be calculated according to the parietal lobe, temporal lobe, parietal lobe, and temporal lobe process, and the results will be stored in the cache until the end of the loop.
4. Enter the results in the cache into the parietal lobe to get the final result.
5. The loop size can be corrected for the loss of the last calculation.

Note1: In Algorithm 3, if both **parietalLobe** and **temporalLobe** use ERIR, the algorithm is called the SADS algorithm. If **parietalLobe** uses ERIR and **temporalLobe** uses MemERIR, the algorithm is called the SADSMem algorithm.

From the processing flow of the SADS or SADSMem algorithm, it can be seen that we have incorporated the following mechanisms into the initial System1 and System2 mechanisms to enhance network performance.

ERIR network Mechanism: By introducing the ERIR algorithm, the system significantly increased the maximum sequence length it can handle, while effectively reducing the consumption of computing and memory resources, thereby achieving the optimization of the overall network performance.

Dual-system iterative FIFO communication mechanism of CacheBuff: The CacheBuff is designed to serve as a shared working memory and processing result between System 1 (the processing system) and System 2 (the memory system). It adopts the First-In-First-Out (FIFO) replacement strategy. When the cache is full, the earliest data are automatically discarded to maintain dynamic updates. This mechanism enables System 1 and System 2 to access each other's outputs and perform iterative refined analysis and result correction, forming a closed-loop for collaborative processing.

memory networks mechanism: To effectively capture stable patterns and unchanged information within the data, we have incorporated a memory network component with persistent memory capabilities. Due to its unique structural design, recurring key information is solidly stored in the memory ERIR network, forming a stable knowledge base. When needed, the ERIR network can flexibly call upon these memory information by querying CacheBuff or directly accessing it to assist in processing the current sequence.

The specific SADS algorithm is presented in Algorithm 3.

In Algorithm 3, **mLen**, **sNum** and **crossRate** represent the maximum segmentation length of parameters(maxLen), the number of

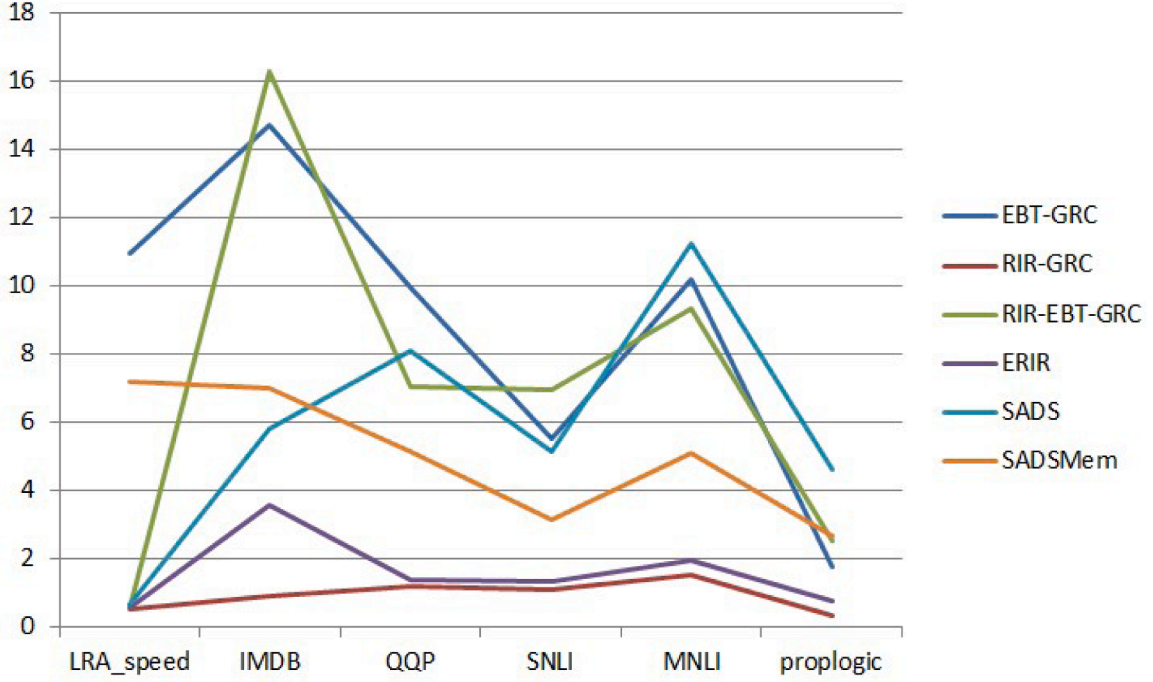


Fig. 12. GPU memory curve.

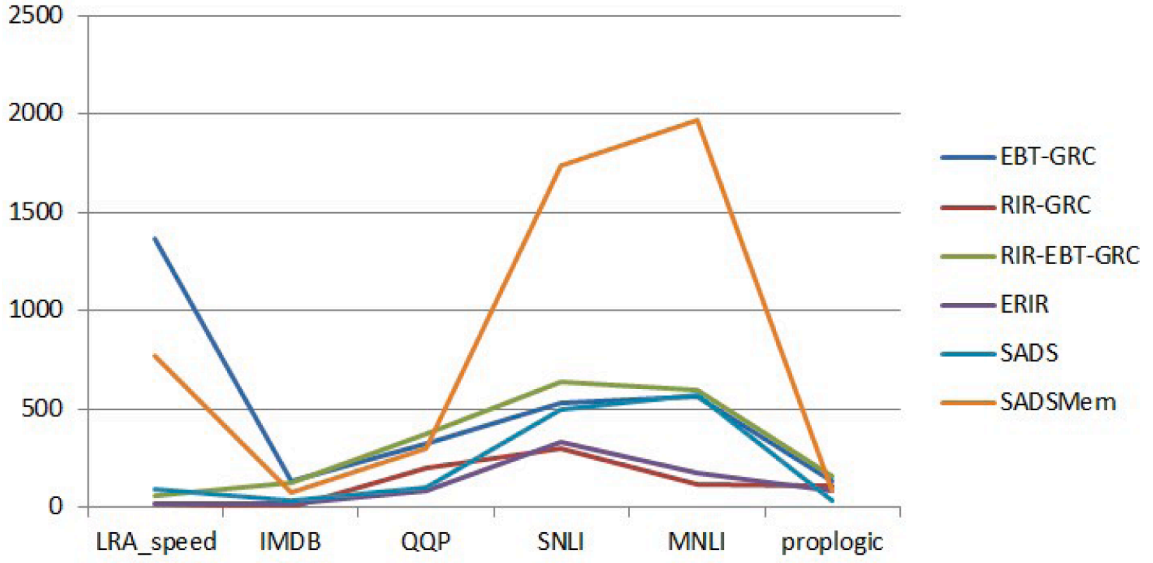


Fig. 13. Execution time curve.

sequence segments per layer(splitNum) and the ratio of interaction length between adjacent sequences(crossRate) in the ERIR algorithm, respectively. The parameter **mloop** represents the maximum number of interactions between the **frontalLobe** and the **temporalLobe**. The **cacheSize** indicates the maximum capacity of the cache. In steps 2 - 5, all components of the SADS model are initialized. In steps 7 - 22 define a computing loop of top-occipital-temporal interaction. This loop begins with the initial calculation of the **parietalLobe**, and its output is processed interactively and passed to the **temporalLobe** for deep analysis. Finally, the analysis results flow back to the **parietalLobe** to complete

the calculation loop and generate the final result. In step 23, add the residual network calculation results to the original calculation results.

7. Experimental results

To verify the effectiveness of the algorithms, this paper draws on the method in Jishnu and Cornelia (2023a) to conduct performance comparison tests on the proposed ERIR, SADS, and SADSMem algorithms, as well as the algorithms introduced in Jishnu and Cornelia (2023a) and the Mamba algorithm.

Algorithm 3 (SADS): Calculate $y = Sads(x)$.

Input: x is a input tensor of the form

1: $[batch, xNum, xDim]$

Input: $hiddenDim > 0$

Input: $mLen > 0, sNum > 0, 0 < crossRate < 1$

Input: $mLoop > 0, cacheSize > 0$

2: *parietalLobe* is the ERIR model defined in Section 4.3

3: *temporalLobe* is the ERIR model defined in Section 4.3

4: $outRes \leftarrow Bp(xDim, hDim, labelDim)$

5: A queue *cacheBuff* of length *cacheSize* is created

6: $resRst \leftarrow outRes[x]$

7: $parietalOut \leftarrow parietalLobe[x]$

8: **push** *parietalOut* to *cacheBuff*

9: **while** $index < mLoop$ **do**

10: **if** $index = 0$ **then**

11: $temporalOut \leftarrow temporalLobe[x]$

12: **else**

13: $temporalOut \leftarrow temporalLobe[cacheBuff]$

14: **end if**

15: **push** *temporalOut* to *cacheBuff*

16: **if** $index \neq mLoop - 1$ **then**

17: $parietalOut \leftarrow parietalLobe[cacheBuff]$

18: **push** *parietalOut* to *cacheBuff*

19: **end if**

20: $index \leftarrow index + 1$

21: **end while**

22: $output \leftarrow parietalLobe[cacheBuff]$

23: **return** $output + resRst$

Table 1

Calculate the accuracy of algorithms at LRA speed and IMDB dataset, **Cont.** represent **Contrast Test** from Matt et al. (2020), **Count.** represent **Counterfactual Test** from Divyansh et al. (2020).

Model	LRA speed	IMDB		
	IID	IID	Cont.	Count.
EBT-GRC	36.50 _{19.5}	91.56 _{1.9}	78.48 _{10.4}	86.88 _{9.6}
RIR-GRC	16.15 _{7.3}	90.70 _{2.7}	78.68 _{10.2}	83.19 _{11.8}
RIR-EBT-GRC	18.50 _{20.8}	91.07 _{1.3}	77.86 _{2.5}	86.06 _{9.8}
MAMBA	16.15 _{0.0}	55.33 _{28.0}	50.20 _{0.0}	50.20 _{0.0}
ERIR	52.15 _{8.0}	90.76 _{3.2}	67.21 _{16.7}	83.40 _{11.1}
SADS	52.34 _{0.9}	91.40 _{12.5}	78.89 _{52.9}	85.04 _{36.3}
SADS-MEM	52.34 _{50.9}	90.77 _{10.0}	77.04 _{37.3}	84.22 _{16.7}

Table 2

calculate the accuracy of algorithms at QQP task, $PAWS_{QQP}$ and $PAWS_{Wiki}$ represent data from Yuan et al. (2019).

Model	QQP		
	IID	$PAWS_{QQP}$	$PAWS_{Wiki}$
EBT-GRC	83.67 _{4.0}	37.07 _{10.2}	46.86 _{3.0}
RIR-GRC	84.55 _{3.1}	38.99 _{12.3}	44.35 _{2.0}
RIR-EBT-GRC	84.00 _{2.9}	38.99 _{6.0}	45.91 _{10.1}
MAMBA	50.0 _{0.0}	71.78 _{0.0}	68.49 _{0.0}
ERIR	93.52 _{41.8}	59.82 _{58.4}	52.43 _{11.2}
SADS	84.31 _{52.7}	65.43 _{44.0}	53.72 _{6.1}
SADS-MEM	84.14 _{22.9}	52.88 _{36.6}	53.13 _{11.2}

7.1. Dataset description

•LRA Speed Task: As introduced in Yi et al. (2021), LRA refers to a collection of tasks specifically designed to assess the ability of neural models to handle long-range dependencies. In our evaluation, we focus on the subset of tasks related to language. This subset includes the

Table 3

Calculate the accuracy of algorithms at SNLI task, **Hard** represent data set from Suchin et al. (2018), **Break** represent data set from Max et al. (2018), **Count.** represent data set from Divyansh et al. (2020).

Models	SNLI			
	IID	Hard	Break	Count.
EBT-GRC	84.64 _{3.8}	69.15 _{10.1}	54.37 _{13.6}	58.93 _{6.6}
RIR-GRC	84.83 _{3.2}	69.08 _{9.0}	57.19 _{29.1}	58.87 _{9.0}
RIR-EBT-GRC	85.10 _{0.6}	70.83 _{1.7}	53.55 _{4.4}	59.37 _{3.6}
MAMBA	32.94 _{6.2}	34.80 _{11.3}	87.44 _{355.6}	33.62 _{8.8}
ERIR	92.33 _{2.6}	83.93 _{3.6}	71.93 _{130.7}	77.0 _{11.1}
SADS	91.58 _{17.5}	82.82 _{37.5}	74.14 _{178.6}	78.68 _{33.7}
SADS-MEM	91.64 _{17.0}	83.04 _{36.8}	98.95 _{235.5}	78.43 _{30.5}

ListOps LRA, the Text LRA (specifically, the character-level IMDB task described in Andrew et al. (2011)), and the Retrieval LRA (that is, the character-level document pair matching task).

•IMDB Task:

we choose the binary sentiment classification task IMDB constructed by Andrew et al. (2011) as a training set. Using the IID test set created by Jishnu and Cornelia (2023b) as the test set, the test set completed by Gardner et al. Matt et al. (2020) is selected as the comparison test set. The test data completed by Divyansh et al. (2020) are selected as the counterfactual set.

•QQP Task:

QQP is a task to classify whether two sequences given in a pair are paraphrases or not. As a standard, Zhiguo et al. (2017), we randomly selected 10000 samples for validation and IID test sets. For each partition, 5000 samples are retained as paraphrases, and the other 5000 samples are not retained as paraphrases. We also use the adversarial test sets PAWS QQP and PAWS WIKI proposed by Yuan et al. (2019).

•SNLI Task:

SNLI (Samuel et al., 2015b) is a natural language inference task. This is a 3D classification task that involves identifying inferential relationships between two given sequences. We use the same partitions for the train, development, and IID test set Jishnu and Cornelia (2021). To improve training efficiency, the sequence data with length ≥ 150 will be filtered. We also use additional partitioning of the test set for stress testing. We use the hard test set partitioned by Suchin et al. (2018), the break test set partitioned by Glockner et al. Max et al. (2018), and from the counterfactual test set by Divyansh et al. (2020).

•MNLI Task:

MNLI (Adina et al., 2018) is another NLI data set with a format similar to SNLI. We used the original development data set, both matched and mismatched, as a test set. To improve training efficiency, the sequence data with length ≥ 150 will be filtered. The actual training set is 1000 filtered data points. We use ConjNLI (Swarnadeep et al., 2020) as an additional test dataset and use the stress test dataset developed by Aakanksha et al. (2018).

•Proplogic Task:

Samuel et al. (2015a) first introduced Logical Inference. This task requires classifying fine-grained inferential relationships between two given sequences, and its form is similar to that of formal sentences in propositional logic. Like Ke et al. (2018), we trained our models on data splits where the number of logical connectives is less than or equal to 6. We then present the results obtained from the Out-of-Distribution (OOD) test sets, in which the number of logical connections ranges from 4 to 12. In addition, we adopted the same data splits as those used by Jishnu and Cornelia (2019), Ke et al. (2018), Zhiqiang et al. (2022).

Table 4

Calculate the accuracy of algorithms at MNLI task, **MM**. represents the **Mismatch Dev** set from Adina et al. (2018), **ConjNLI** represents data set from Swarnadeep et al. (2020), **NegM**. represents the **Negation Match** data set from Aakanksha et al. (2018), **NegMM**. represents the **Negation Mismatch** data set from Aakanksha et al. (2018), **LenM**. represents the **Length Match** data set of Aakanksha et al. (2018), **LenMM**. represents the **Length Mismatch** data set from Aakanksha et al. (2018).

Models	MNLI						
	Match	MM	ConjNLI	NegM	NegMM	LenM	LenMM
EBT-GRC	72.20 _{3.1}	71.90 _{0.9}	41.09 _{4.7}	54.64 _{17.5}	55.15 _{20.8}	65.97 _{11.1}	67.10 _{10.7}
RIR-GRC	71.58 _{4.2}	71.76 _{3.2}	41.57 _{13.1}	51.15 _{13.8}	51.49 _{11.9}	58.67 _{13.0}	59.02 _{18.3}
RIR-EBT-GRC	72.15 _{3.0}	72.13 _{0.3}	40.93 _{4.7}	52.87 _{29.7}	53.20 _{29.1}	64.14 _{1.4}	65.48 _{5.9}
MAMBA	35.44 _{0.0}	35.22 _{0.0}	32.74 _{0.0}	35.44 _{0.0}	35.22 _{0.0}	35.44 _{0.0}	35.22 _{0.0}
ERIR	86.09 _{12.1}	86.36 _{16.4}	71.91 _{42.3}	78.06 _{42.8}	78.02 _{44.3}	80.18 _{14.6}	81.07 _{12.6}
SADS	88.00 _{17.1}	88.78 _{17.4}	79.93 _{69.6}	68.09 _{4.9}	67.98 _{6.4}	74.43 _{21.5}	74.87 _{26.0}
SADS-MEM	85.29 _{11.6}	85.31 _{16.7}	71.58 _{45.5}	76.77 _{46.2}	76.52 _{45.1}	79.58 _{3.8}	80.36 _{9.4}

Table 5

Calculate the accuracy of algorithms at Proplogic task, IDs from 5 to 12 represent the average accuracy rates and standard deviations obtained at the number of operations ranging from 5 to 12 respectively.

Models	Proplogic							
	ID5	ID6	ID7	ID8	ID9	ID10	ID1	ID12
EBT-GRC	99.33 _{0.5}	98.67 _{1.5}	98.08 _{1.2}	97.19 _{2.1}	96.50 _{2.0}	94.52 _{6.8}	94.09 _{6.8}	93.08 _{1.4}
RIR-GRC	98.20 _{1.6}	96.71 _{4.3}	94.17 _{3.8}	85.80 _{6.3}	64.03 _{7.6}	47.29 _{11.3}	46.87 _{13.4}	44.66 _{7.4}
RIR-EBT-GRC	99.23 _{1.8}	98.65 _{1.2}	97.79 _{1.7}	92.23 _{4.0}	70.26 _{5.5}	50.20 _{2.9}	51.85 _{21.2}	53.34 _{7.0}
ERIR	98.42 _{61.6}	97.85 _{61.5}	95.58 _{46.5}	92.82 _{42.3}	90.89 _{26.9}	89.54 _{28.8}	87.50 _{35.1}	87.57 _{11.4}
SADS	88.08 _{7.8}	87.41 _{9.5}	87.78 _{5.5}	87.57 _{4.3}	87.39 _{4.3}	87.18 _{14.5}	85.87 _{12.8}	86.63 _{12.9}
SADS-MEM	98.05 _{41.9}	97.78 _{39.7}	98.32 _{46.3}	98.26 _{42.8}	98.87 _{49.2}	99.09 _{51.5}	99.30 _{58.3}	99.17 _{55.8}

Table 6

Algorithm evaluation results, Ind_i Value represents the evaluation result of the **Method i**, and Ind_i Rank represents the evaluation ranking of the Ind_i , where $i = 1, 2, 3$.

Models	Algorithm evaluation results					
	Ind_1 Value	Ind_1 Rank	Ind_2 Value	Ind_2 Rank	Ind_3 Value	Ind_3 Rank
EBT-GRC	319.3	4	137.3	4	74.1	4
RIR-GRC	555.2	6	267.9	6	65.0	6
RIR-EBT-GRC	501.3	5	247.4	5	67.1	5
ERIR	103.3	2	34.9	2	82.4	2
SADS	168.6	3	38.6	3	79.9	3
SADS-MEM	56.1	1	11.0	1	84.2	1

Table 7

GPU memory usage situation.

Models	LRA Speed	IMDB	QQP	SNLI	MNLI	Proplogic	AVER.
EBT-GRC	10.9	14.7	9.9	5.5	10.1	1.7	8.8
RIR-GRC	0.5	0.9	1.2	1.1	1.5	0.3	0.9
RIR-EBT-GRC	0.5	16.2	7.0	6.9	9.3	2.5	7.1
ERIR	0.5	3.5	1.4	1.3	1.9	0.7	1.6
SADS	0.6	5.8	8.0	5.1	11.2	4.6	5.9
SADS-MEM	7.1	6.9	5.1	3.1	5.0	2.6	5.0

Table 8

Execution time.

Models	LRA Speed	IMDB	QQP	SNLI	MNLI	Proplogic
EBT-GRC	1368	134	325	535	564	134
RIR-GRC	19	1	203	298	119	108
RIR-EBT-GRC	61	126	372	643	594	161
ERIR	15	20	85	336	175	87
SADS	89	32	99	502	575	38
SADS-MEM	771	79	297	1740	1971	85

7.2. Test environment information

We tested all algorithms in the following environment:

Hardware: 18 vCPU + RTX 4090D(24GB GPU memory) + 755GB memory

Software: Ubuntu 22.04

7.3. Experimental instructions

All experiments are repeated three times with different random seeds and the average results are reported in this section. The subscript represents the standard deviation. As an example, $90_1 = 90 \pm 0.1$.

7.4. Dataset test results

We compare ERIR, SADS and SADSMem with EBT-GRC Jishnu and Cornelia (2023a), RIR-GRC Jishnu and Cornelia (2023a), RIR-EBT-GRC Jishnu and Cornelia (2023a), Mamba Albert and Tri (2024) and obtain the following tables.

In LRA speed and IMDB task:

Performing accuracy tests on each algorithm using the LRA speed and the IMDB task, we obtained Table 1.

In QQP task:

Performing accuracy tests on each algorithm using the QQP task, we obtained Table 2.

In SNLI task:

Performing accuracy tests on each algorithm using the SNLI task, we obtained Table 3.

In MNLI task:

Performing accuracy tests on each algorithm using the MNLI task, we obtained Table 4.

In Proplogic task:

Performing accuracy tests on each algorithm using the Proplogic task, we obtained Table 5.

7.5. Dataset test analysis

In this paper, the following three methods are used to compare the accuracy of different algorithms.

Method 1. Calculate the sum of the differences between the optimal results in each data set and the results obtained by the corresponding algorithms.

Method 2. Calculate the variance of the differences between the optimal results in each data set and the results obtained by the corresponding algorithms.

Method 3. Calculate the average classification accuracy of each algorithm in each data set.

According to the evaluation methods and the data in Table 1, Table 2, Table 3, Table 4, Table 5, the evaluation results of Table 6 are obtained.

Based on the results of the analysis in Table 6, we get the following test analysis results.

1. Compared to other algorithms, the RIR, SADS, and SADSMem algorithms have obviously advantages in Ind_1 , Ind_2 , and Ind_3 .

2. When there is the limitation on GPU memory size, considering the limitation imposed by the number of hidden layer neurons, the classification results of Mamba neurons are much lower than those of the RIR or ERIR algorithms.

7.6. GPU Memory usage analysis

In this experiment, the GPU video memory sizes (GB) used by each algorithm are shown in Table 7. A curve graph is plotted based on the results in Table 7, and the results are shown in Fig. 12.

Based on the results of the analysis in Table 7, and Fig. 12, we get the following results of the test analysis.

1. As can be seen in Fig. 12, the average GPU memory usage size of the ERIR and RIR-GRC algorithms is much smaller than that of other algorithms.

2. Since the SADS and SADSMem algorithms use two ERIR as parts of the parietal lobe and temporal lobe, respectively, their memory footprints are significantly larger than those of the ERIR algorithm.

7.7. Execution time analysis

In the experiment, the same training and testing methods as those in the reference Jishnu and Cornelia (2023a) are used, the execution time (minute) used by each algorithm is shown in Table 8 and Fig. 13.

Based on the results of the analysis in Fig. 13, we obtain the following results.

1. As can be seen in Fig. 13, the time consumption of the ERIR is basically the same as that of other algorithms.

2. Since the SADS and SADSMem algorithms use two ERIR as parts of the parietal lobe and temporal lobe, respectively, their time consumption is significantly larger than those of the ERIR algorithm.

8. Conclusion

In this paper, we introduce the idea of analyzing the stability of neural networks without calculating gradients. Through this idea, we can investigate the stability of the neural network before training. Thus, we explore the advantages and disadvantages of the RIR network and make improvements, including the ERIR network and the MemERIR networks, respectively. Based on the above two improvements, we present the SADS model based on the external memory network and fast and slow circulation of the brain and achieve good results.

In addition, the following questions are worth discussing for a neural network.

1. The theoretical discussions on the generalization upper bound of neural networks and the VC dimension have been given in the previous research results. However, it is not possible to tell from these conclusions how individual components in the network affect the generalization results. Therefore, it is necessary to discuss how to obtain the influence of each component of the neural network algorithm on the generalization results through theoretical derivation.

2. The current method of selecting part of the data set for testing is used to improve the reliability of the network. However, this method relies heavily on the distribution of the dataset itself and therefore can only say whether the neural network is reliable under that particular distribution. In practice, the data distribution cannot be assumed to be consistent with the experimental data distribution. Therefore, methods for effective testing of neural networks need to be discussed.

CRedit authorship contribution statement

Fanglue Liu: Validation, Software, Investigation, Formal analysis, Conceptualization; **Ting Qian:** Writing – review & editing, Writing – original draft, Supervision, Project administration, Funding acquisition; **Yanhong She:** Writing – review & editing, Writing – original draft, Supervision, Resources, Project administration.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The authors would like to thank all reviewers for all constructive and helpful review comments that have helped to improve the paper significantly.

The authors thank the developers of the source models for their contributions and for making their work available. (Corresponding authors: Ting Qian and Yanhong She).

This work is supported by National Natural Science Foundation of China (Nos. 12471442, 12171388), the Scientific Research Program Funded by Education Department of Shaanxi Provincial Government (Youth Innovation Team of Shaanxi Universities) (No. 25JP140) and Shaanxi Provincial Natural Science Basic Research Program – Key Projects – Youth Project (Category A) (2026JC-JCQN-015).

Code availability

we have shared the relevant code and the entire project on GitHub.

1. The simulation results of the properties of the activation function mentioned in this paper refer to <https://github.com/liufanglue/ActivateFunc>.

2. The corresponding ERIR testing project has been completed in <https://github.com/liufanglue/RecurrentRnn>, and the specific class name is **RecurrentRnnNetwork**.

3. The corresponding SADS testing project has been completed in <https://github.com/liufanglue/Sads>, and the specific class name is **Sad-sNetwork**.

4. The corresponding SADS-Mem testing project has been completed in <https://github.com/liufanglue/SadsMem>, and the specific class name is **SadsNetwork**.

5. Other comparison test items have been completed. The address is <https://github.com/liufanglue/SadsOtherTest>.

It should be noted that, due to the different characteristics of the algorithms, there may be subtle differences in the implementation of classes with the same name in different projects. The basic code and data framework are cited from Jishnu and Cornelia (2023a).

References

- Aakanksha, N., Abhilasha, R., Norman, S., Carolyn, R., & Graham, N. (2018). Stress test evaluation for natural language inference. *27th international conference on computational linguistics*.
- Adina, W., Nikita, N., & Samuel, B. (2018). A broad-coverage challenge corpus for sentence understanding through inference. *2018 conference of the North American chapter of the association for computational linguistics: Human language technologies*.
- Albert, G., Karan, G., Ankit, G., & Christopher, R. (2022). On the parameterization and initialization of diagonal state space models. *Advances in Neural Information Processing Systems*, 35.
- Albert, G., & Tri, D. (2024). Mamba: Linear-time sequence modeling with selective state spaces. <https://doi.org/10.48550/arXiv.2312.00752>
- Andrew, L., Raymond, E., Peter, T., Dan, H., Andrew, Y., & Christopher, P. (2011). Learning word vectors for sentiment analysis. *49th annual meeting of the association for computational linguistics: Human language technologies*.
- Bouchereau, M., Chartier, P., Lemou, M., & ehats, F. (2023). Machine learning methods for autonomous ordinary differential equations. *arXiv preprint arXiv:2304.09036*.
- Couellan, N. (2021). The coupling effect of lipschitz regularization in neural networks. *SN Computer Science*, 2, 113. <https://doi.org/10.1007/s42979-021-00514-x>
- Couellan, N. (2022). Learning smooth neural functions via lipschitz regularization. *International conference on computer graphics and interactive techniques, SIGGRAPH*.
- Daniel, K. (Ed.) (2011). Thinking, fast and slow. New York: Farrar, Straus and Giroux.
- Divyansh, K., Eduard, H., & Zachary, L. (2020). Learning the difference that makes a difference with counterfactually-augmented data. *International conference on learning representations*.
- Gouk, H., Pfahringer, B., & Cree, M. J. (2021). Regularisation of neural networks by enforcing lipschitz continuity. *Machine Learning*, 110, 393–416.
- Guido, M., Razvan, P., Kyunghyun, C., & Yoshua, B. (2014). On the number of linear regions of deep neural network. *Neural Information Processing Systems, NIPS*.
- Jishnu, C., & Cornelia, C. (2019). Modeling hierarchical structures with continuous recursive neural networks. *38th international conference on machine learning*.
- Jishnu, C., & Cornelia, C. (2023a). Recursion in recursion: Two-level nested recursion for length generalization with scalability. *Conference on neural information processing systems, NeurIPS*.
- Jishnu, R., & Cornelia, C. (2021). Modeling hierarchical structures with continuous recursive neural networks. *38th international conference on machine learning*.
- Jishnu, R., & Cornelia, C. (2023b). Beam tree recursive cells. *40th international conference on machine learning*.
- Ke, T., Arianna, B., & Christof, M. (2018). The importance of being recurrent for modeling hierarchical structure. *2018 conference on empirical methods in natural language processing*.
- Lee, S., Lee, J., & Park, S. (2020). Lipschitz-certifiable training with a tight outer bound. *Advances in Neural Information Processing Systems*, 33, 16891–16902.
- Lei, M., Zhang, M., & Wang, K. (2026). Research on bidding optimization strategy for virtual power plants with wind-solar-storage systems based on IGD-TDRO. *Electrical Engineering*, 108, 208. <https://doi.org/10.1007/s00202-026-03544-x>
- Lv, X., Cui, S., Wang, Y., Lu, J., Yu, P., & Wang, K. (2026). Patch time series transformer-based short-term photovoltaic power prediction enhanced by artificial fish. *Energies*, 19, 284. <https://doi.org/10.3390/en19010284>
- Matt, G., Yoav, A., Victoria, B., Jonathan, B., Ben, B., Sihao, C., Pradeep, D., Dheeru, D., Yanai, E., Ananth, G., Nitish, G., Hannaneh, H., Gabriel, I., Daniel, K., Kevin, L., Jiangming, L., Nelson, F., Phoebe, M., Qiang, N., Sameer, S., Noah, A., Sanjay, S., Reut, T., Eric, W., Ally, Z., & Ben, Z. (2020). Evaluating models' local decision boundaries via contrast sets. *Association for computational linguistics: EMNLP*.
- Max, G., Vered, S., & Yoav, G. (2018). Breaking NLI systems with sentences that require simple lexical inferences. *56th annual meeting of the association for computational linguistics*.
- Pintore, M., & Després, B. (2022). Learning smooth neural functions via lipschitz regularization. *International conference on computer graphics and interactive techniques, SIGGRAPH*.
- Pintore, M., & Després, B. (2024). Computable lipschitz bounds for deep neural networks. *arXiv preprint arXiv:2410.21053*
- Samuel, R., Christopher, M., & Christopher, P. (2015a). Tree-structured composition in neural networks without tree-structured architectures. *2015th international conference on cognitive computation*.
- Samuel, R., Christopher, P., & Christopher, D. (2015b). A large annotated corpus for learning natural language inference. *2015 conference on empirical methods in natural language processing*.
- Suchin, G., Swabha, S., Omer, L., Roy, S., Samuel, B., & Noah, A. (2018). Annotation artifacts in natural language inference data. *2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*.
- Sukhbaatar, S., Szlam, A., Weston, J., & Fergus, R. (2015). End-to-end memory networks. *Advances in Neural Information Processing*, 28.
- Sukhbaatar, S., Szlam, A., Weston, J., & Fergus, R. (2019). Ordered memory. In *Advances in Neural Information Processing Systems*. (vol. 32).
- Suzanne, C. (Ed.) (2018). Permanent Present Tense: The Man with No Memory, and What He Taught the World. Bei Jing: Beijing Times Chinese Press.
- Swarnadeep, S., Yixin, N., & Mohit, B. (2020). ConjNLI: Natural language inference over conjunctive sentences. *2020 conference on empirical methods in natural language processing (EMNLP)*.
- Szegedy, C., Zaremba, W., Sutskever, I., Bruna, J., Erhan, D., Goodfellow, I., & Fergus, R. (2014). Intriguing properties of neural networks. *International conference on learning representations, ICLR*.
- Taha, M. A. (2025). Logarithmic memory networks (LMNs): Efficient long-range sequence modeling for resource-constrained environments. *arXiv preprint arXiv:2501.07905*.
- Terrence, T. (Ed.) (2012). Topics in Random Matrix Theory. 201 Charles Street Providence, Rhode Island: American Mathematical Society.
- Virmaux, A., & Scaman, K. (2018). Lipschitz regularity of deep neural networks: Analysis and efficient estimation. *Advances in Neural Information Processing Systems*, 31.
- Walter, R. (Ed.) (2020). Functional Analysis. Beijing: China Machine Press.
- Weng, L., Zhang, H., Chen, H., Song, Z., Hsieh, C., Daniel, L., Boning, D., & Dhillon, I. (2018). Towards fast computation of certified robustness for reLU networks. *35th international conference on machine learning, ICML*.
- Weston, J., Chopra, S., & Bordes, A. (2015). Memory networks. <https://doi.org/10.48550/arXiv.1410.3916>
- Weston, J., Chopra, S., & Bordes, A. (2016). Recurrent memory network for language modeling. *North American chapter of the association for computational linguistics*.
- Xuezhe, M., Chunting, Z., Xiang, K., Junxian, H., Liangke, G., Graham, N., Jonathan, M., & Luke, Z. (2023). Mega: Moving average equipped gated attention. In *the eleventh international conference on learning representations*.
- Yi, T., Mostafa, D., & Samira, A., et al. (2021). Long range arena : A benchmark for efficient transformers. *arXiv preprint arXiv:2011.04006*.
- Yuan, Z., Jason, B., & Luheng, H. (2019). Paws: Paraphrase adversaries from word scrambling. *2019 conference of the North American chapter of the association for computational linguistics: Human language technologies*.
- Yuechen, L., Zishun, W., Chen, Q., & Zongben, X. (2026). Gate: Adaptive learning with working memory by information gating in multi-lamellar hippocampal formation. *PLoS ONE*, 22, 6. <https://doi.org/10.1371/journal.pcbi.1014438>
- Zhiguo, W., Wael, H., & Radu, F. (2017). Bilateral multi-perspective matching for natural language sentences. *26th international joint conference on artificial intelligence*.
- Zhiqiang, S., Zechun, L., & Eric, X. (2022). Sliced recursive transformer. *Computer vision-ECCV 2022: 17th European conference*.